MODULE 02 · BLOCK 1 - COMPUTER PROGRAMMING AND PYTHON FUNDAMENTALS

Literals, variables and numeral systems

How a value's spelling decides its type, four ways to write a number, and the rules for naming things.

BLOCK 1 of 4 EXAM WEIGHT 18% MODULE 02 of 28

admin@sthithapragna.com

A literal is a value written directly in code. **How it is spelled** decides its type, before your program does anything with it.

What the spelling tells Python

LITERAL	TYPE	WHAT DECIDES IT
`42 -7 1_000_000`	int	Digits only, no point, no E
`3.14 .5 4. 4e2`	float	A decimal point, or an E exponent
`"42" 'hi' ""`	str	Quotes, whatever is inside them
`True False`	bool	Exactly these two words, capitalised
`None`	NoneType	The single value meaning no value

THE TRAP

4e2 is a float and "42" is a string. Neither looks like it, and both are favourite exam traps.

type() tells you, and shows it as a class

```
1  print(type(42))       # <class 'int'>
2  print(type(42.0))     # <class 'float'>
3  print(type(4e2))      # <class 'float'>
4  print(type("42"))     # <class 'str'>
5  print(type(True))     # <class 'bool'>
6  print(type(None))     # <class 'NoneType'>
```

3 An E exponent always makes a float, even when the value is whole.

4 Quotes win. The digits inside do not make it a number.

Output is written as <class '...'>. Older material shows <type '...'>, which is Python 2.

E means times ten to the power

```
1  print(3E8)          # 300000000.0
2  print(4.2e-3)       # 0.0042
3  print(+42E7)        # 420000000.0
4  print(0.0001)       # 0.0001
5  print(0.00001)      # 1e-05
6  print(1e16)         # 1e+16
```

1-3 3E8 means 3 times 10 to the 8. Upper or lower case E, always a float.

5-6 Python chooses how to print a float. Very small or very large values come back in E notation.

The exponent must be a whole number: 1E2.5 is a SyntaxError.

Underscores inside numbers

```
1  salary = 1_250_000      # fine: 1250000
2  mask = 0b1111_0000      # fine in any base
3  _1000 = 5               # a variable name, not a number
4  bad = 1__000            # SyntaxError
5  worse = 1000_           # SyntaxError
```

1-2 One underscore between digits is ignored. It only helps the reader.

3 Starting with an underscore makes it a name, not a number.

4-5 Two underscores in a row, or one at the end, is invalid.

The same kind of number, four ways to write it

SYSTEM	BASE	PREFIX	DIGITS ALLOWED	EXAMPLE
Binary	2	0b	0 and 1	0b1010 is 10
Octal	8	0o	0 to 7	0o17 is 15
Decimal	10	none	0 to 9	42 is 42
Hexadecimal	16	0x	0 to 9, A to F	0x1F is 31

ON THE EXAM

The prefix only changes how you write the number. It is still an int, and print shows it in decimal.

Place values: each column is worth base times the next

```
# binary 0b1101:  1*8  + 1*4 + 0*2 + 1*1  = 13
# octal  0o17:    1*8  + 7*1              = 15
# hex    0x1F:    1*16 + 15*1             = 31
print(0b1101, 0o17, 0x1F)      # 13 15 31
```

1 Binary columns, right to left: 1, 2, 4, 8, 16. Add the columns holding a 1.

3 In hex, F is 15. Multiply each digit by its column value and add.

Invalid digits are SyntaxErrors: 0b102, 0o8. And 017 is rejected: use 0o17 for octal.

bin(), oct() and hex() return strings

```
1  print(bin(10))          # 0b1010
2  print(oct(8))           # 0o10
3  print(hex(255))         # 0xff
4  print(type(hex(255)))   # <class 'str'>
5  print(0XFF, 0B11)       # 255 3
```

1-3 Each function gives the text form, prefix included, lower case letters.

4 The result is a string, not a number. You cannot do arithmetic on it.

bool and None

- **True** and **False** are the only bool values. In arithmetic they act as 1 and 0.
- So True + True is 2, and True * 10 is 10.
- **None** is the single value of type NoneType. It means no value.
- None is not 0, not False and not an empty string. None == 0 is False.

> ON THE EXAM
>
> None can be assigned, printed and compared. It cannot be used in arithmetic.

Strings, briefly, before module 15

```
print("Python" == 'Python')   # True
print("Py" "thon")            # Python
print("4" + "2")              # 42
print(4 + 2)                  # 6
```

1. Single and double quotes make exactly the same string.
2. Two string literals side by side are joined into one.
3. Plus on strings joins them. The digits never become numbers.

Rules for a legal name

- Letters, digits and underscores only. Any length.
- **Cannot start with a digit.** 2nd_place is a SyntaxError.
- Cannot be one of the 35 keywords.
- **Case-sensitive:** total, Total and TOTAL are three different names.
- No spaces or hyphens. first-name reads as first minus name.

> ON THE EXAM
>
> Letters include non-English ones: cafe with an accent is a legal name. A leading underscore is legal.

Legal names versus good names

THE RULES

Break them: SyntaxError

- No leading digit
- No keywords
- Only letters, digits, _
- Checked by the interpreter

PEP 8 CONVENTIONS

Break them: still runs

- snake_case for variables
- UPPER_CASE for constants
- CapWords for class names
- Checked only by people and tools

THE TIEBREAKER

If a question asks whether a name is legal, apply only the rules. MyVar is ugly but legal.

A variable is a name stuck on a value

```
1  x = 10          # the name x now refers to 10
2  x = "ten"       # x moves to a string. Types follow values
3  print(x)        # ten
4  y = x           # y refers to the same value as x
5  print(count)    # NameError: count is not defined
```

1-2 Assignment creates the name on first use. The type belongs to the value, not the name.

5 Using a name before assigning it is a NameError, raised when the line runs.

Chained and multiple assignment

```
1  a = b = c = 0          # chained: three names, one value
2  x, y = 1, 2            # multiple: x is 1, y is 2
3  x, y = y, x            # swap in one line
4  print(a, b, c, x, y)   # 0 0 0 2 1
```

1 Chained assignment gives every name the same value.

3 The whole right side is evaluated first, then the names are bound. That is why the swap works.

The counts must match: x, y = 1, 2, 3 raises ValueError: too many values to unpack.

What to carry into the exam

- A point or an E makes a float. Quotes make a str. **4e2 is 400.0.**
- 0b, 0o, 0x change the spelling only. The value is an int, printed in decimal.
- bin(), oct() and hex() return **strings**.
- Names: no leading digit, no keywords, case-sensitive. PEP 8 is style, not law.
- In a, b = x, y the right side is evaluated completely before binding.

Which types do distance, rate and code have?

```
1  distance = 2_500
2  rate = 4E2
3  code = "42"
```

A int, int, str

B int, float, str

C str, float, int

D The code raises a SyntaxError

◆ PAUSE AND DECIDE

Which types do distance, rate and code have?

```
1  distance = 2_500
2  rate = 4E2
3  code = "42"
```

type(rate) is <class 'float'>; its value is 400.0.

A int, int, str

4E2 is whole in value, but an E exponent always makes a float.

B int, float, str

Underscore is fine, E means float, quotes mean str.

C str, float, int

The underscore does not make a string, and quotes never make an int.

D The code raises a SyntaxError

One underscore between digits is legal since Python 3.6.

THE SIGNAL

Read the spelling, not the value. E or a point: float. Quotes: str.

What is the output of this program?

```
1  a = 0b1010
2  b = 0o17
3  c = 0x1F
4  print(a + b + c)
```

A 58

B 46

C 56

D 0b10100o170x1F

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = 0b1010
2  b = 0o17
3  c = 0x1F
4  print(a + b + c)
```

10 + 15 + 31 = 56

A **58**

Reads 0o17 as decimal 17. Octal 17 is 8 + 7 = 15.

B **46**

Reads 0x1F's first digit as 10s. In hex, the 1 is worth 16.

C **56**

8 + 2 = 10, then 8 + 7 = 15, then 16 + 15 = 31.

D **0b10100o170x1F**

These are ints, not strings. Plus adds them.

THE SIGNAL

Convert each literal with place values, then do ordinary arithmetic.

Each line is run on its own. Which two are legal assignments? (Choose two.)

```
1  _total = 1
2  2nd_place = 2
3  first-name = 3
4  Class = 4
```

A Line 1

B Line 2

C Line 3

D Line 4

◆ PAUSE AND DECIDE

Each line is run on its own. Which two are legal assignments? (Choose two.)

```
1  _total = 1
2  2nd_place = 2
3  first-name = 3
4  Class = 4
```

A Line 1

A name may start with an underscore.

B Line 2

A name cannot start with a digit. SyntaxError.

C Line 3

The hyphen is a minus sign. You cannot assign to first - name.

D Line 4

class is a keyword, but Class is not. Case matters.

THE SIGNAL

Check each name against three rules: first character, allowed characters, keywords with exact case.

What is the output of this program?

```
1  a = b = 5
2  a, b = b + 1, a * 2
3  print(a, b)
```

A 6 12

B 6 10

C 5 10

D The code raises a SyntaxError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = b = 5
2  a, b = b + 1, a * 2
3  print(a, b)
```

Right side first: (5 + 1, 5 * 2) is (6, 10). Then a and b are bound.

A `6 12`

Updates a first, then uses the new a. Python does not.

B `6 10`

Both expressions use the old values, 5 and 5.

C `5 10`

Treats the first assignment as skipped. Both names change.

D The code raises a SyntaxError

Chained and multiple assignment are both legal.

THE SIGNAL

In multiple assignment, the whole right side is evaluated before any name changes.

What happens when this program runs?

```
1  Total = 10
2  total = Total + 5
3  print(total)
4  print(TOTAL)
```

A 15 is printed twice

B 15 is printed, then 10

C Nothing is printed; the program raises a SyntaxError

D 15 is printed, then the program raises a NameError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  Total = 10
2  total = Total + 5
3  print(total)
4  print(TOTAL)
```

NameError: name 'TOTAL' is not defined. Did you mean: 'Total'?

A. 15 is printed twice
Treats total and TOTAL as one name. Names are case-sensitive.

B. 15 is printed, then 10
Treats TOTAL as Total. It is a third, separate name.

C. Nothing is printed; the program raises a SyntaxError
The grammar is fine. The problem only appears when line 4 runs.

D. 15 is printed, then the program raises a NameError
TOTAL was never assigned. Line 3 has already printed 15.

THE SIGNAL

Different case, different name. An unknown name is a runtime NameError, so earlier lines have run.

> A physics program needs the value thirty point one one times ten to the power nine, written as a single float literal.

Which literal writes this value correctly?

A `30E11.9`

B `30.11*10^9`

C `30.11E9.0`

D `30.11E9`

◆ PAUSE AND DECIDE

Which literal writes this value correctly?

A `30E11.9`

The exponent must be a whole number. SyntaxError.

B `30.11*10^9`

Not a literal. ^ is bitwise XOR, so this raises a TypeError on a float.

C `30.11E9.0`

Same problem: a decimal point in the exponent is a SyntaxError.

D `30.11E9`

Mantissa 30.11, exponent 9. Its value is 30110000000.0.

THE SIGNAL

The part after E is always a whole number. And ^ in Python is XOR, never a power.